Chapter 8

Optimization of Queueing Systems

8.1 Introduction

We have built a decent understanding of queueing systems by analyzing the variants derived from the memoryless property with changes in the arrival rates and the number of servers. In real life, every queueing system is tied to some resource that needs to be optimized: the staff scheduling of a retail store, the server capacity of an online application, the number of active stations serving a manufacturing process. As mentioned in the previous Chapter, the matching of resources needs to make economic sense in the context of the business goal: satisfiying some target demand within a prescribed time, sustaining a level of service for all incoming customers, etc.

8.2 Linear Allocation

Going back to the staff scheduling models studied in 4 imagine that you want to schedule a facility that serves customers, each taking a random time τ in service. This setting is typically a call center, a retail store, a restaurant, a factory or any system where when jobs accumulate past certain level they wait in queue. The resource is an integer number representing the number of servers available to serve customers: such as employees. The rate the system serves is μ.

It would be great if there is a system where the service $\mathsf{E}(W)$, say the waiting time a customer spends in the system: waiting in queue plus their service time behaves linearly. The simplest model is a single server, operating at a rate μ. In this case, we can apply the same logic: a customer arriving sees in average $\mathsf{E}(N)$ customers, each served in average at a time $1/\mu$ per customer, plus their own service time $1/\mu$. Then, a random customer waits in this system as:

$$\mathsf{E}(W) = \mathsf{E}(N)(1/\mu) + 1/\mu, \tag{8.1}$$

But we also know that by Little's law that $\mathsf{E}(N) = \mathsf{E}(a)\mathsf{E}(W)$, replacing this in the equation yields $\mu\mathsf{E}(W) = \mathsf{E}(a)\mathsf{E}(W) + 1$ or:

$$\mathsf{E}(W) = \frac{1}{\mu - \mathsf{E}(a)}, \tag{8.2}$$

The implicit assumption is that the system is stable, that is, that the arrival rate $\mathsf{E}(a)$ is less than the processing rate of the server μ. This expression is great because it is almost linear on the server term μ. Then, we could approximate the rate of the server μ to be proportional to the number of resources, say $x\tilde{\mu}$, choosing $\tilde{\mu}$ in such way that it matches the desired processing rate. Where x is a vector choosing the number of resources assigned to a given collection of schedules such as in Chapter 4, this leads to the following scheduling formulation:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & \mu A x^\intercal \geq \mathsf{E}(\boldsymbol{A}), \\ & 1 \leq (\tilde{\mu} A x^\intercal - \mathsf{E}(\boldsymbol{A}))\bar{W}, \\ & x \in \mathcal{X}. \end{aligned} \tag{8.3}$$

The first constraint just represents the stability condition of the system, ensuring that the demand $\mathsf{E}(\boldsymbol{A})$, which is a vector with elements $\mathsf{E}(a(t-1,t))$ is below the processing capacity of the system. The second constraint represents limiting the average waiting time to some maximum level $\bar{W}$, such that, at all hours the service is in average below this level. As before, $f(x)$ can be choosen to be the payroll. Or even a more elaborate measure, we illustrate this with an example:

Example 8.1 (Car Wash Scheduling)**.** An owner of a car wash wants to minimize their operating costs which are proportional to their payroll, expressed as a vector $cx^\intercal$ and the water and electricity costs of their car wash, which are γ dollars per unit of time when each station is on (there are in total K stations). While servicing all the demand, the owner wants to do a scheduling at minimum cost. How to model this problem?

First, satisfying all the demand is the constraint $\mu A x^\intercal \geq \mathsf{E}(\boldsymbol{A})$, which is the same stability condition of the queueing system. The cost part is more elaborated: the payroll cost is $cx^\intercal$ as in the Scheduling chapter, but the variable cost is in principle a complex calculation as it would require in principle estimating the average behaviour of the system for different staffing configurations $Ax^\intercal$. Then, we want to know how to express $f(x) = cx^\intercal + \text{Variable Cost}$. To do this, we can use what we learned in the Chapter 7 via Little's law, we know that the average number in system is given by:

$$\mathsf{E}(N(t)) = \mathsf{E}(a(t-1,t))\mathsf{E}(W) = \mathsf{E}(a(t-1,t))\frac{1}{\tilde{\mu}(Ax^\intercal)_t - \mathsf{E}(a(t-1,t))}, \tag{8.4}$$

Then, the cost over the periods the car-wash is open $t = 1, 2, \ldots, T$ is approximately $\sum_{t=1}^{T} \gamma \mathsf{E}(N(t))$. Then, the next step is to linearize the variable cost part of the objective $\sum_{t=1}^{T} \gamma \mathsf{E}(N(t))$ on x so that we can use an optimizer to solve the problem. A way to write this problem as a MIP is to estimate $\mathsf{E}(N(t))$ for the range of possible values it can take, for example, let $\mathsf{E}_k(N(t)) = \frac{\mathsf{E}(a(t-1,t))}{\tilde{\mu}k - \mathsf{E}(a(t-1,t))}$ and then creating a variable $z_{k,t}$ selecting the corresponding value to 1 if $(Ax^\intercal)_t = k$. Then, the problem can be written as:

$$
\begin{aligned}
\min \quad & cx^\intercal + \gamma \sum_{t=1}^{T} \mathsf{E}_k(N(t)) z_{k,t} && (8.5)\\
\text{s.t.} \quad & \mu A x^\intercal \geq \mathsf{E}(\boldsymbol{A}), \\
& (Ax^\intercal)_t = \sum_{k=1}^{K} k z_{k,t}, \text{ for } t = 1, \ldots, T, \\
& \sum_{k=1}^{K} z_{k,t} = 1 \text{ for } t = 1, \ldots, T, \\
& x \in \mathcal{X}, z_{i,k} \in \{0, 1\}.
\end{aligned}
$$

Clearly, this type of formulation has its pros and cons: on one hand it is a tractable formulation that in a single pass allows to solve a seemingly complex problem without multiple simulation runs in a difficult combinatorial space. On the other hand, it is still an approximation on many levels.

The first element, is that $\tilde{\mu}$ should be chosen carefully as one mega-server working always at a rate $k\mu$ is not the same as k servers at a rate μ. To see why, when there is less than k jobs in the system, say $i < k$ the superserver operates at a rate $k\mu$ while the multiple servers operate at a rate $\min(i, k)\mu$. Then, it makes sense to deteriorate the rate $\tilde{\mu}$ to be slower to compensate for the speedup. A simple way is to define a factor $\tilde{\mu}_k = \eta_k \mu$ where $\eta_k < 1$, thus slowing down the server. One such value for η could be the probability that all servers are active $\eta_k = P(i \geq k) = \sum_{i=k}^{\infty} \pi_i$ in the original $M/M/k$ queue, this slows down the server to reflect that when $i < k$ some servers are idle in the original model, thus making the approximation better reflect the original behavior of k servers into a single one with normalized rate $\mu\eta_k$.

8.2.1 Mean-variance optimization

Another common feature throughout out exposition is modeling the risk-aware version of the problem. In this case, we can exploit the same approximation to consider the variance of the number in system, that is $\mathsf{Var}_k(N(t))$ with multiplier λ (see Chapter 3 to see different ways to choose the parameter). This leads to the equivalent risk aware formulation:

$$
\begin{aligned}
\min \quad & cx^{\intercal} + \gamma \sum_{t=1}^{T} \mathsf{E}_k(N(t)) z_{k,t} + \lambda \sum_{t=1}^{T} \mathsf{Var}_k(N(t)) z_{k,t} \\
\text{s.t.} \quad & \mu A x^{\intercal} \geq \mathsf{E}(\boldsymbol{A}), \\
& (Ax^{\intercal})_t = \sum_{k=1}^{K} k z_{k,t}, \text{ for } t = 1, \ldots, T, \\
& \sum_{k=1}^{K} z_{k,t} = 1 \text{ for } t = 1, \ldots, T, \\
& x \in \mathcal{X}, z_{i,k} \in \{0, 1\}.
\end{aligned} \tag{8.6}
$$

Closed expressions for the Variance of queueing systems are widely known, for example, for the $M/M/1$ approximation, we have $\mathsf{Var}_k(N(t)) = \frac{\mathsf{E}(a(t-1,t))k\tilde{\mu}}{[k\tilde{\mu} - \mathsf{E}(a(t-1,t))]^2}$.

8.3 Accuracy and non-exponential service times

The approach presented is compelling in the sense that it allows to model service times at determistic epochs $t, t+1, ...$ that are useful for planning horizons where the accuracy is tolerable (this is true for many manufacturing systems where noise averages out) and the resources scheduled have a difficult combinatorial nature (for example, scheduling people).

There are other systems that have a mathematically more amenable description of their resource space but require more accuracy. One key example of such system are computer services that deal with thousands of client request simultaneously. Large Language Models (LLMs) providers have to solve a server allocation problem to service their demand within a reasonable time in system for hundreds of thousands of concurrent requests. Here, our understanding of queueing systems shines to solve these kind of problems. Imagine the following problem:

$$
\begin{aligned}
\min \quad & \mathsf{E}\left(\int_0^T [c(k_t) + \gamma(N_t)] dt \right) \\
s.t. \quad & \mathsf{P}(W > s) \leq \alpha.
\end{aligned} \tag{8.7}
$$

Where k_t is the number of active servers at time t, N_t is the number in system at time t and c and $c(k_t)$ is a cost function dependent on the number of active servers, while $\gamma(N_t)$ is a variable cost function depending on the number of queries in the system. W represents the random amount of time in system the customers face. The problem can be stated as finding the number of servers that in average keeps the system at minimum cost while with a low probability α makes that the time in system does not exceed a threshold s. This could easily represent the problem that a cloud provider has to solve to service an application or service (such as providing inference for LLMs). In principle we could apply the same strategy that we proposed in the exposition of the previous section, but in this case we can do better.

Recalling Chapter 4 we can apply a similar dynamic programming principle, were we can solve the problem using the same iterative procedure. Start with a set of actions, the number of servers, that is, $k = 0, 1, \ldots, K$ and discretize the time horizon at intervals of size Δt. The key as always is to set up a recursive relation for the problem, in this case:

$$V_t(N) = \min_k [c(k, N)\Delta t + V_{t+1}(f(N, k, r_t))], \tag{8.8}$$

Where $r_t := \mathsf{E}(a(t, t + \Delta t))$ and $f(N, k, r_t)$ represents the transition function of a system that currently has N customers, k active servers and a expected arrival rate r_t. The first and easiest part is defining the loss function $c(k, N)$ which is just $c(k) + \gamma(N)$.

For the dynamics of the system $f(N, k, r_t)$, we can exploit what we know about the transient behaviour of a queueing system. For a small Δt we have that in expected value:

$$N_{next} = f(N, k, r_t) = \begin{cases} N + (r_t - k\mu)\Delta t & \text{if } N \geq k \quad \text{(Linear/Saturated)} \\ Ne^{-\mu\Delta t} + \frac{r_t}{\mu}(1 - e^{-\mu\Delta t}) & \text{if } N < k \quad \text{(Exponential/Recovery)} \end{cases} \tag{8.9}$$

Similar to the previous chapters, a way to incorporate the service constraint $\mathsf{P}(W > s) \leq \alpha$ is to prune the paths where this occurs. The trick is to express this probability in terms of the primitives of the model.

There are many ways to achieve this, perhaps the simplest is the argument that we used to derive the waiting time of the single server queue. In this case, when the server is saturated $N > k$, the waiting time a new customer faces is the waiting time of people in queue $(N - k)$, each entering service at a rate $k\mu$. And after that, the customer enters their own service at a rate μ. That is, the time the N-th customer stays in the system with k active servers is in average:

$$\mathsf{E}(W(N)) = \frac{(N - k)^+}{k\mu} + \frac{1}{\mu}, \tag{8.10}$$

Likewise, its variance is the variance of the exponential times, which are independent from each other, that is:

$$\mathsf{Var}(W(N)) = \frac{(N - k)^+}{(k\mu)^2} + \frac{1}{\mu^2}, \tag{8.11}$$

Then, we can approximate the random waiting time at each step as:

$$W(N) \approx \frac{\max(N, k)}{k\mu} + Z\sqrt{\frac{\max(N, k)}{(k\mu)^2}}, \tag{8.12}$$

Therefore, the probability $\mathsf{P}(W(N) > s) \leq \alpha$ or $\mathsf{P}(W(N) \leq s) \geq 1 - \alpha$, is given by the expression:

$$sk\mu \geq \max(N, k) + \sqrt{\max(N, k)}\Phi^{-1}(1 - \alpha), \tag{8.13}$$

Then, combinations of N, k that violate this condition are not admissible in the DP iteration.

8.3.1 Non-exponential service times

Throughout our exposition we have considered exponential service times. The exponential service times have allowed us to use the memoryless property and analyze the system at each snapshot of time. Considering non-exponential service times tends to make control problems intractable as there is a need to keep track of either the elapsed or remaining service times as they are no longer memory-less. The reason is that the reliability function in general does not follow time-homogeneity, that is, $R(s)R(t) \neq R(s+t)$.

In recent years, a new framework called QPLEX (see [5]) allows to estimate transition dynamics without doing full simulations or having to keep track of an explosive state-space. The key insight is that given a specific number of customers in the system we sample a single customer in service remaining time (say $N_t = N$, that in the previous subsection we were taking as the expected value). Conditional on N the remaining time in service can be seen as an i.i.d. random variable. Call L as the random remaining service time of a job picked at random. Estimating the distribution of L for a given time t is what QPLEX does through a series of clever tricks. Call this distribution ν_t. With this distribution as input, conditioning on a starting number of jobs N_t we can express most of the primitives of the problem. For example, the number of expected departures is $N_t\nu_t(L = \Delta t|N_t)$. In fact, QPLEX estimates the distributions of the number in system at the next step p_t and the remaining service durations ν_t, call these pair of distributions p_t, ν_t.

QPLEX is able to approximate these sequential distributions through clever conditionings. Using the distributions p_t, ν_t we can redo our dynamic programming procedure as:

$$V_t(N) = \min_k [\mathsf{E}_{p_t}(c(k,N))\Delta t + \mathsf{E}_{\nu_t}(V_{t+1}|N,k,r_t)]. \tag{8.14}$$